BULLPEN ALPHA: High-Frequency Data Pipeline

Architectural Solution for Ingesting and Visualizing 1,000,000+ Ticks/Second

Bullpen Alpha is a distributed systems project designed to address the Firehose Problem: maintaining system stability and UI responsiveness while processing high-velocity data streams. The pipeline demonstrates the practical application of binary serialization, message brokerage, and temporal data aggregation.

System Architecture

The application employs a decoupled architecture to isolate high-throughput ingestion from stateful UI rendering.

- Ingestion Layer: Python 3.13 utilizing the confluent-kafka client for high-performance production.
- Serialization Layer: Google Protocol Buffers (Protobuf) for minimal binary payloads.
- Brokerage Layer: Apache Kafka and Zookeeper (Containerized) for distributed queuing and fault tolerance.
- Aggregation Layer: Node.js backend acting as a temporal sampler and WebSocket gateway.
- Visualization Layer: React 19 with Tailwind CSS v4, utilizing a high-density bento-style layout for data-rich environments.

Engineering Implementation

1. Binary Serialization Efficiency

To maximize network bandwidth, the system implements Protocol Buffers instead of standard JSON. This optimization reduced the data packet size by approximately 75% (from an average of 85 bytes to 21 bytes per message), significantly lowering the I/O overhead on the Kafka broker.

2. Producer Buffer Management

The Python producer manages memory during high-velocity spikes (1M+ events/second) through a non-blocking poll(0) loop and a recursive BufferError handler. This ensures the producer remains stable under heavy backpressure from the Kafka broker without dropping data or crashing the host process.

3. Temporal Aggregation (Downsampling)

To prevent client-side performance degradation, the Node.js aggregator implements a 100ms heartbeat (10Hz). By buffering and summarizing Kafka batches before emitting them via WebSockets, the system maintains a stable 60 FPS UI framerate while the backend processes millions of individual events in the background.

4. Bidirectional Control Loop

The system includes a full-stack Circuit Breaker. A command initiated in the React UI travels through the WebSocket tunnel to the Node.js server, which programmatically executes consumer.pause() or consumer.resume() on the Kafka stream. This demonstrates real-time, bidirectional state management across different runtimes.

Performance Benchmarks

Metric	Performance Data
Peak Throughput	1,000,000+ ticks/sec
Payload Weight	21 Bytes (Binary/Protobuf)
End-to-End Latency	< 200ms
UI Rendering	60 FPS Stable

Deployment Instructions

1. Infrastructure: Initialize Kafka/Zookeeper via **docker-compose up -d.**
2. Aggregation Server: Navigate to /server and execute **node index.js.**
3. Client Dashboard: Navigate to /client and execute **npm run dev.**
4. Data Producer: Navigate to /producer and execute **python main.py.**

Project by Aaish Abbas Zaidi

Second-Year B.Tech Computer Science (Data Science) @ JSSUN Specializing in Scalable Systems & Machine Learning Pipelines